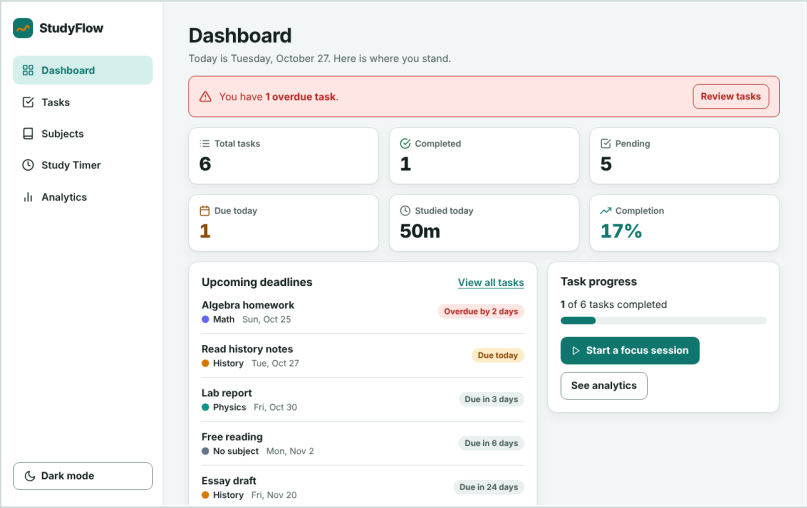
StudyFlow

Student Productivity Dashboard

Project guide: how it works, how it is built, how it was tested

Written for beginners learning React

StudyFlow is a web app that helps a student plan tasks, organise subjects, run focused study sessions with a Pomodoro timer, and see where their time goes. It runs entirely in the browser and saves everything on the user's own device.



The StudyFlow dashboard on a desktop screen (real screenshot).

At a glance	
Built with	React 19, JavaScript (ES modules), HTML5, CSS3, Vite 8, LocalStorage. No other runtime libraries.
Size	5 pages, 35 reusable components, 4 custom hooks, 11 utility modules (about 3,400 lines of source).
Tests	74 unit tests (all pass). 28 automated browser tests (all pass, on the production build and on the dev server).
Production build	276 KB of JavaScript (84 KB compressed) plus 18 KB of CSS.

Contents

1. What StudyFlow does 4

2. Technologies and why they were chosen 5

Why so few dependencies? . . . 5

3. Project architecture 6

Folder structure . . . 6

Following one action through the app . . . 7

Key design decisions . . . 7

4. React concepts used (with real code) 8

Components and props . . . 8

useState: remembering things . . . 8

Never change state directly . . . 8

useEffect: reacting to changes and cleaning up . . . 8

useMemo, useCallback, useRef . . . 9

Context: sharing data . . . 9

Custom hooks . . . 9

Controlled forms and validation . . . 9

Lists and stable keys . . . 10

Conditional rendering and empty states . . . 10

Two patterns you may not have seen . . . 10

5. JavaScript concepts used 11

Dates: the part that usually goes wrong . . . 11

6. File-by-file reference 12

Entry and pages . . . 12

State and hooks . . . 12

Logic (utils/) . . . 13

Components . . . 13

7. How each feature works 14

7.1 Task management . . . 14

7.2 Search, filters, sorting . . . 14

7.3 Deadlines . . . 15

7.4 Subjects . . . 15

7.5 LocalStorage and safe loading . . . 15

7.6 Dashboard statistics . . . 16

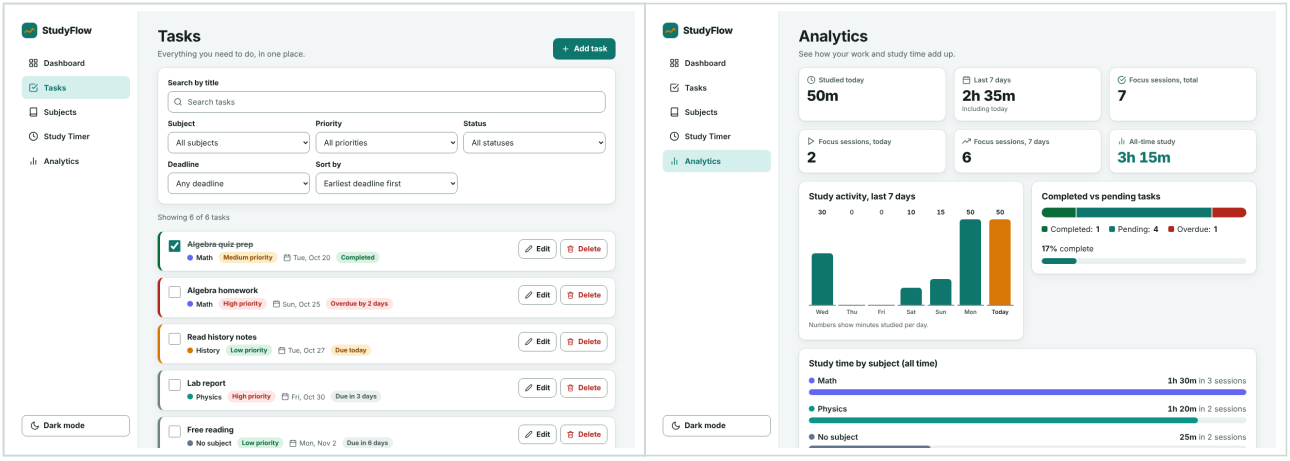
7.7 The Pomodoro timer . . . 16
7.8 Study tracking . . . 17
7.9 Analytics . . . 17
7.10 Light and dark themes . . . 18
7.11 Responsive design . . . 18
7.12 Accessibility . . . 19
8. Testing and quality **20**
8.1 What the browser tests verify . . . 21
8.2 Errors found, and how each was solved . . . 21
8.3 Manual testing checklist . . . 22
9. Run, publish and deploy **24**
9.1 Installation and running . . . 24
9.2 Optional: browser tests . . . 24
9.3 Git and GitHub . . . 24
9.4 Deployment . . . 24
10. Known limitations **26**
Ideas for next steps . . . 26
11. Presenting it, and a glossary **27**
How to talk about this project . . . 27
Glossary . . . 27

1. What StudyFlow does

A student has three recurring problems: remembering what is due, deciding what to work on, and knowing whether they are actually studying. StudyFlow gives one place for each of these.

Page	What the student can do
Dashboard	See total, completed and pending tasks, tasks due today, time studied today, completion percentage, and the next deadlines. A red banner appears when something is overdue.
Tasks	Add, edit and delete tasks with a title, description, subject, priority (High, Medium, Low) and deadline. Tick tasks done or pending. Search by title; filter by subject, priority, status and deadline; sort by deadline; clear everything with one button.
Subjects	Add, edit and delete subjects, each with a colour. Duplicate names are refused. Deleting a subject never deletes its tasks.
Study Timer	A Pomodoro timer: focus 25 min, short break 5 min, long break 15 min (all changeable). Start, pause, resume and reset. Choose the subject you study. Finished focus sessions are recorded once, and shown in a history.
Analytics	Completed vs pending vs overdue tasks, study time today and this week, a seven-day activity chart, study time by subject, and counts of completed focus sessions.

Across all pages: light and dark themes (remembered), a sidebar on desktop and a bottom navigation bar on phones, forms that explain mistakes, confirmation before anything is deleted, and friendly messages whenever there is nothing to show yet.



Left: the Tasks page with search, filters and colour-coded deadlines. Right: Analytics.

> **Where the numbers come from**
>
> No statistic in StudyFlow is typed in or stored. Every number on the Dashboard and Analytics pages is calculated, each time, from the real list of tasks and the real list of study sessions. If you delete a task, the totals change by themselves.

2. Technologies and why they were chosen

Technology	What it is	How StudyFlow uses it
HTML5	The structure of a web page.	Semantic elements: `main`, `nav`, `aside`, `header`, `section`, `ul`, `time`, `dl`, `dialog`, `fieldset`, `label`. They give screen readers and keyboards meaning for free.
CSS3	The look and layout of the page.	CSS variables for colours (so dark mode is one switch), Grid and Flexbox for layout, media queries for screen sizes. No CSS framework.
JavaScript	The programming language of the browser.	Modern ES modules: `import` / `export`, arrow functions, spread, destructuring, optional chaining.
React	A library for building the screen out of small reusable pieces (components).	Function components and hooks only. React redraws the right parts of the screen whenever data changes.
Vite	A build tool and development server.	`npm run dev` shows changes instantly. `npm run build` produces small optimised files for hosting.
LocalStorage	A small key/value store inside the browser.	Saves tasks, subjects, sessions, preferences and the running timer, so data survives a refresh.
Node test runner	Testing built into Node.js.	Runs the 74 unit tests with `npm test`. Nothing extra to install.

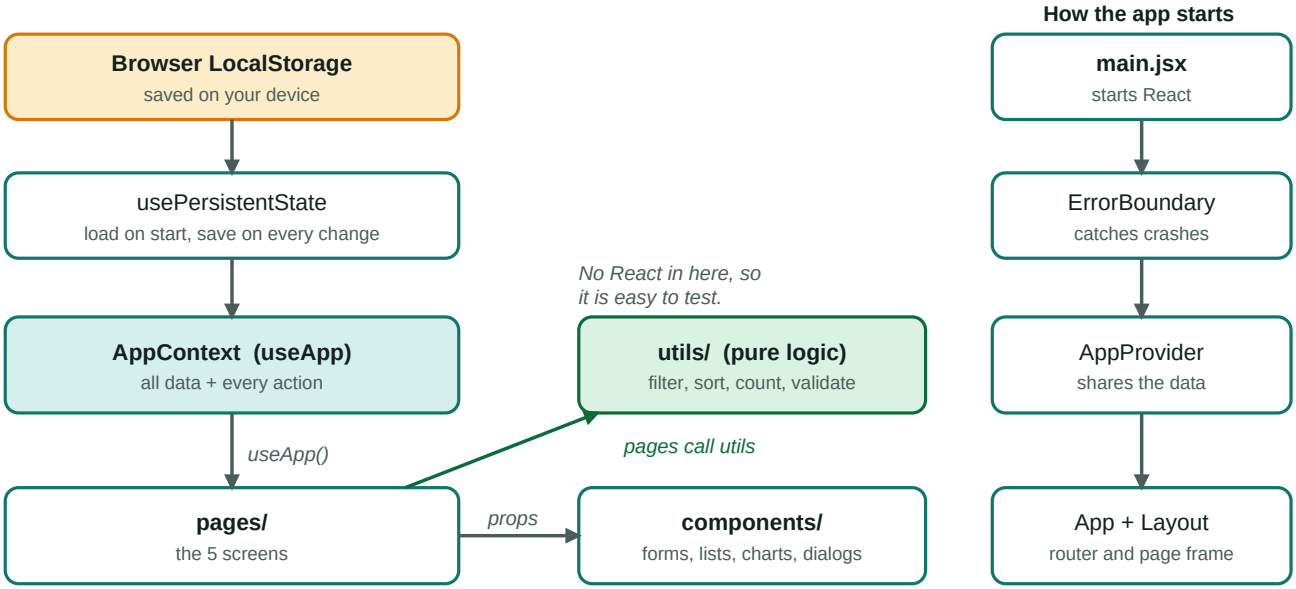
Why so few dependencies?

The project asked to avoid unnecessary dependencies. The runtime needs only `react` and `react-dom`. Instead of a router, state library, chart library, icon pack or date library, StudyFlow uses about 200 lines of its own code for each need. The trade-off is honest: a real company project would often choose a mature library for routing or charts. Here, writing the small version yourself is how you learn what those libraries do.

Instead of...	StudyFlow uses...
React Router	The URL hash (`#/tasks`) and a 10-line hook, `useHashRoute`. Works on any static host.
Redux / Zustand	React Context + `useState`, in `AppContext.jsx`.
A chart library	Plain HTML and CSS bars (`BarChart`, `SubjectBars`, `TaskSplit`), each with a hidden text table for screen readers.
An icon library	A small set of inline SVG icons in `Icon.jsx`.
date-fns / Moment	Date helpers in `dates.js`, using simple "YYYY-MM-DD" text.

3. Project architecture

Architecture means "where does each kind of code live, and how do the parts talk to each other?". StudyFlow has three layers, and a rule that keeps them tidy: **code only depends on the layers below it.**



The three layers

- **Logic (`utils/`).** Plain JavaScript functions that take data in and return results. They know nothing about React or the browser screen. Example: `filterAndSortTasks(tasks, filters, today)`. Because they are "pure" (same input, same output, no side effects), they are easy to test.
- **Data and state (`context/`, `hooks/`).** Holds the application data, saves it, and provides actions like `addTask`. This is the only layer that changes data.
- **Screen (`pages/`, `components/`).** Reads data, shows it, and calls actions when the user clicks. Components contain almost no rules; they ask `utils/` and the context.

Folder structure

```
studyflow/
  index.html            the one HTML page (also sets the saved theme before React starts)
  vite.config.js        Vite settings
  package.json          project name, scripts, dependencies
  public/favicon.svg
  src/
    main.jsx            entry point: starts React
    App.jsx             picks the page to show from the URL
    context/AppContext.jsx      all shared data + actions
    hooks/                      usePersistentState, usePomodoro, useToday, useHashRoute
    utils/                      constants, dates, validation, tasks, study, timer,
                                storage, sanitize, format, id, routes
    components/                 35 reusable pieces (Modal, TaskForm, StatCard, BarChart ...)
    pages/                      Dashboard, Tasks, Subjects, StudyTimer, Analytics
    styles/                     theme.css, base.css, layout.css, components.css
  tests/                74 unit tests for utils
  e2e/                  28 browser tests (Python + Playwright, optional)
```

Following one action through the app

The best way to understand the architecture is to follow a click. Here is what happens when you tick the checkbox next to a task:

Step	What happens	Where
1	You click the checkbox. The `onChange` handler runs `onToggle(task.id)`.	`TaskItem.jsx`
2	`onToggle` was passed down as a prop. It is really `toggleTask` from the context.	`Tasks.jsx` -> `TaskList.jsx` -> `TaskItem.jsx`
3	`toggleTask` builds a NEW tasks array with that one task flipped (never editing the old array) and calls `setTasks`.	`AppContext.jsx`
4	React notices the data changed and re-runs the components that use it.	React
5	`usePersistentState` sees the new array and writes it to LocalStorage.	`usePersistentState.js`
6	The Dashboard (next time you open it) calls `computeTaskStats`, which counts the new situation. Nothing was updated by hand.	`Dashboard.jsx`, `utils/tasks.js`

Key design decisions

Decision	Reason
One Context for all data	Many pages need the same tasks and subjects. Context avoids passing props through many layers ("prop drilling").
The timer lives in the Context, not on the Timer page	The countdown must keep running when you visit other pages. A page's state is lost when you leave it.
Deadlines are text like "2026-10-27"	They save to LocalStorage as they are, sort correctly as text, and avoid time-zone surprises (a deadline is a calendar day, not a moment).
Statistics are calculated, never stored	Stored numbers can disagree with the real data. Calculated numbers cannot.
Deleting a subject keeps its tasks	Silently deleting a student's work would be a bad surprise. Tasks become "No subject".
Data from LocalStorage is "sanitised"	Saved data may be old, hand-edited or damaged. `sanitize.js` rebuilds clean objects and drops what cannot be saved.
Hash routing (`#/tasks`)	No server setup is needed, so it works on GitHub Pages, Netlify or a USB stick.

4. React concepts used (with real code)

Every example below is taken from StudyFlow, so you can open the file and see it in context.

Components and props

A **component** is a function that returns what should appear on screen (written in JSX, which looks like HTML). **Props** are the inputs, like function arguments. A component should display data and report clicks; it should not contain business rules.

```
// components/StatCard.jsx - one number with a label
export default function StatCard({ label, value, icon, tone = 'neutral', hint }) {
  return (
    <div className={`stat-card stat-card--${tone}`}>
      <dt className="stat-card__label"><Icon name={icon} /> <span>{label}</span></dt>
      <dd className="stat-card__value">{value}</dd>
    </div>
  );
}
// used as:  <StatCard label="Pending" value={stats.pending} icon="tasks" />
```

useState: remembering things

`useState` gives a component memory. It returns the current value and a function to change it. Changing it makes React redraw.

```
// pages/Tasks.jsx
const [filters, setFilters] = useState(DEFAULT_FILTERS);
const updateFilters = (patch) => setFilters({ ...filters, ...patch });
```

Never change state directly

React only notices a change when it receives a **new** object or array. Changing the old one in place (`tasks.push(x)`, `task.completed = true`, `tasks.sort()`) is a classic bug. StudyFlow always builds a new copy with spread, `map` or `filter`:

```
// context/AppContext.jsx
const addTask = (values) =>
  setTasks((current) => [...current, newTask]);                // spread: new array
const deleteTask = (id) =>
  setTasks((current) => current.filter((t) => t.id !== id));    // filter: new array
const toggleTask = (id) =>
  setTasks((current) =>
    current.map((t) => (t.id === id ? { ...t, completed: !t.completed } : t)));
// utils/tasks.js - sort a COPY, never the original
return [...matches].sort(compareTasks);
```

The unit tests include a check that sorting and adding sessions never modify the original array.

useEffect: reacting to changes and cleaning up

`useEffect` runs code after the screen updates, for things outside React: saving, timers, the page title. The function it returns is the **cleanup**, which runs before the next effect or when the component disappears.

```
// hooks/usePersistentState.js
useEffect(() => {
  setSaveFailed(!saveValue(key, value));   // save whenever `value` changes
}, [key, value]);                          // the "dependency list"

// hooks/useToday.js - re-check the date every 30 s, so "overdue" updates after midnight
useEffect(() => {
  const id = setInterval(check, 30 * 1000);
  return () => clearInterval(id);          // cleanup: stop the interval
}, []);
```

useMemo, useCallback, useRef

- `useMemo` remembers a calculated result until its inputs change. The Dashboard uses it so statistics are not recalculated on every redraw: `useMemo(() => computeTaskStats(tasks, today), [tasks, today])`.
- `useCallback` remembers a function, so it keeps the same identity between redraws. Used for context actions so that effects depending on them do not re-run needlessly.
- `useRef` is a box that keeps a value without causing a redraw, or points at a real HTML element. `Layout.jsx` uses it to move keyboard focus to the main area, and `usePomodoro` uses it to remember which finished timer was already handled.

Context: sharing data

```
// context/AppContext.jsx
const AppContext = createContext(null);
export function AppProvider({ children }) { /* load data, define actions ... */
  return <AppContext.Provider value={value}>{children}</AppContext.Provider>;
}
export function useApp() { return useContext(AppContext); }

// any component, any depth:
const { tasks, addTask, today } = useApp();
```

Custom hooks

A custom hook is a function whose name starts with `use` and that uses other hooks. It packages reusable behaviour. `usePersistentState` is `useState` plus saving to LocalStorage, and it is reused for tasks, subjects, sessions, preferences and the timer.

Controlled forms and validation

In a **controlled input**, React owns the value: the input shows `values.title`, and each keystroke calls `setValues`. On Save, `validateTask` (a pure function in `utils/validation.js`) returns `{ errors, values }`. If there are errors they are shown next to the fields and focus moves to the first bad field; if not, the cleaned values are saved.

```
<input value={values.title} onChange={(e) => setValues({ ...values, title: e.target.value })} />
const { errors, values: clean } = validateTask(values, { subjects });
if (Object.keys(errors).length > 0) { setErrors(errors); return; }
onSubmit(clean);
```

Lists and stable keys

When you show a list with `map`, each item needs a `key` that is unique and **never changes** for that item. StudyFlow uses the item's generated id, never the position in the list. With position-based keys, deleting or inserting an item would make React confuse items and show checked boxes on the wrong rows.

```
{tasks.map((task) => <TaskItem key={task.id} task={task} ... />)}
```

Conditional rendering and empty states

```
{tasks.length === 0 ? <EmptyState title="No tasks yet" ... />
 : deadlines.length === 0 ? <EmptyState title="You are all caught up" ... />
 : <DeadlineList tasks={deadlines} today={today} />}
```

Two patterns you may not have seen

- **Render props** (`FormField.jsx`): a component receives a function as its child and calls it with the accessibility attributes the input needs (`id`, `aria-invalid`, `aria-describedby`). This guarantees every field is correctly linked to its label and error message.
- **Error boundary** (`ErrorBoundary.jsx`): a component that catches crashes in the screens below it and shows a helpful message instead of a blank page. It is the only class component in the project, because React requires a class for this feature.

5. JavaScript concepts used

Concept	Example from the project	Why it matters
`map`, `filter`, `reduce`, `some`, `find`	`sessions.reduce((sum, s) => sum + s.durationMin, 0)`	The everyday tools for turning a list into another list or a number. All statistics are built from them.
Spread `...` and destructuring	`{ ...task, completed: true }`, `const { tasks, today } = useApp()`	Copy objects and arrays without changing the original; pull values out neatly.
Optional chaining `?.` and `??`	`subjects.find(...)?.name ?? "No subject"`	Safely handle "might not exist" without crashing.
Template literals	`` `Due in ${diff} days` ``	Build text from values.
Pure functions	`computeTaskStats(tasks, today)`	Same input gives the same output with no side effects, so they are easy to test and trust.
`try` / `catch`	Every LocalStorage read and write in `storage.js`	Storage can fail (private mode, full disk). The app must not crash.
Regular expressions	`/^(\d{4})-(\d{2})-(\d{2})$/` in `dates.js`	Check that a date has the right shape before using it.
`Map`	Grouping study minutes by subject in `study.js`	A dictionary that keeps insertion order and takes any key.
ES modules	`import` / `export` in every file	Splits code into small files with clear dependencies.
`Date.now()` and timestamps	`remaining = endAt - Date.now()`	The foundation of the accurate timer.

Dates: the part that usually goes wrong

Dates are the most common source of bugs in this kind of app. StudyFlow avoids most of them by following a few rules, each backed by a unit test:

- A deadline is stored as text, `"2026-10-27"`: a calendar day with no time and no time zone.
- `isValidDateKey` rejects `"2026-02-30"` (right shape, impossible day) by building a `Date` and checking nothing "rolled over" to the next month. It accepts `"2028-02-29"` (a real leap day).
- Adding days uses `setDate`, which handles month ends, year ends and daylight-saving changes. Counting days between dates uses UTC, so a day is always exactly 24 hours.
- "Today" is the **local** calendar day. A study session that finished at 23:59 counts for that day, not the next one.
- `useToday` re-checks the date every 30 seconds, so if you leave the app open past midnight, "due today" turns into "overdue" by itself (this is tested with a fake clock).

6. File-by-file reference

What each file is for. Read the files in this order if you want to understand the project: `main.jsx`, `App.jsx`, `AppContext.jsx`, one page, then its components.

Entry and pages

File	Purpose
`index.html`	The single page. A tiny script sets the saved theme before React loads, so there is no white flash in dark mode.
`main.jsx`	Starts React and wraps the app in `ErrorBoundary` and `AppProvider`.
`App.jsx`	Looks at the current route and shows the matching page inside `Layout`.
`pages/Dashboard.jsx`	Six summary cards, upcoming deadlines (soonest 6), progress bar, overdue warning.
`pages/Tasks.jsx`	Owns the filter state; combines search, filters and sort; opens the add/edit dialog and the delete confirmation.
`pages/Subjects.jsx`	Add/edit form, list of subjects with counts, delete confirmation that explains tasks are kept.
`pages/StudyTimer.jsx`	Timer dial and controls, subject picker, today's total, timer lengths, session history.
`pages/Analytics.jsx`	Study summary cards, 7-day chart, completed-vs-pending split, study time by subject.

State and hooks

File	Purpose
`context/AppContext.jsx`	Loads and saves all data. Provides tasks, subjects, sessions, preferences, theme, the timer, toast messages and every action to change them.
`hooks/usePersistentState.js`	`useState` that loads from and saves to LocalStorage, and reports damaged or unsaveable data.
`hooks/usePomodoro.js`	Connects the pure timer functions to React: refreshes the clock, detects the end, records the session once.
`hooks/useToday.js`	Returns today's date and updates at midnight.
`hooks/useHashRoute.js`	Returns the current page name from the URL hash.

Logic (`utils/`)

File	Purpose
`constants.js`	Priorities, the 10 subject colours, limits (title 100 characters, etc.), timer defaults and ranges, storage keys.
`dates.js`	Date keys, validation, adding days, counting days, "Due in 3 days" wording.
`validation.js`	Rules for the task, subject and timer-length forms. Includes duplicate-name detection.
`tasks.js`	`getTaskState`, `filterAndSortTasks`, `computeTaskStats`, `getDeadlineList`.
`study.js`	Daily, weekly and per-subject study time, from the list of sessions.
`timer.js`	Start, pause, resume, reset, complete; the long-break rule; `addSessionOnce` (duplicate protection).
`storage.js`	Safe read/write of LocalStorage; keeps a `.backup` copy of data it cannot understand.
`sanitize.js`	Rebuilds clean tasks, subjects, sessions, preferences and timer from untrusted saved data.
`format.js`, `id.js`, `routes.js`	Text formatting ("1h 25m", "25:00"), unique ids, the list of pages.

Components

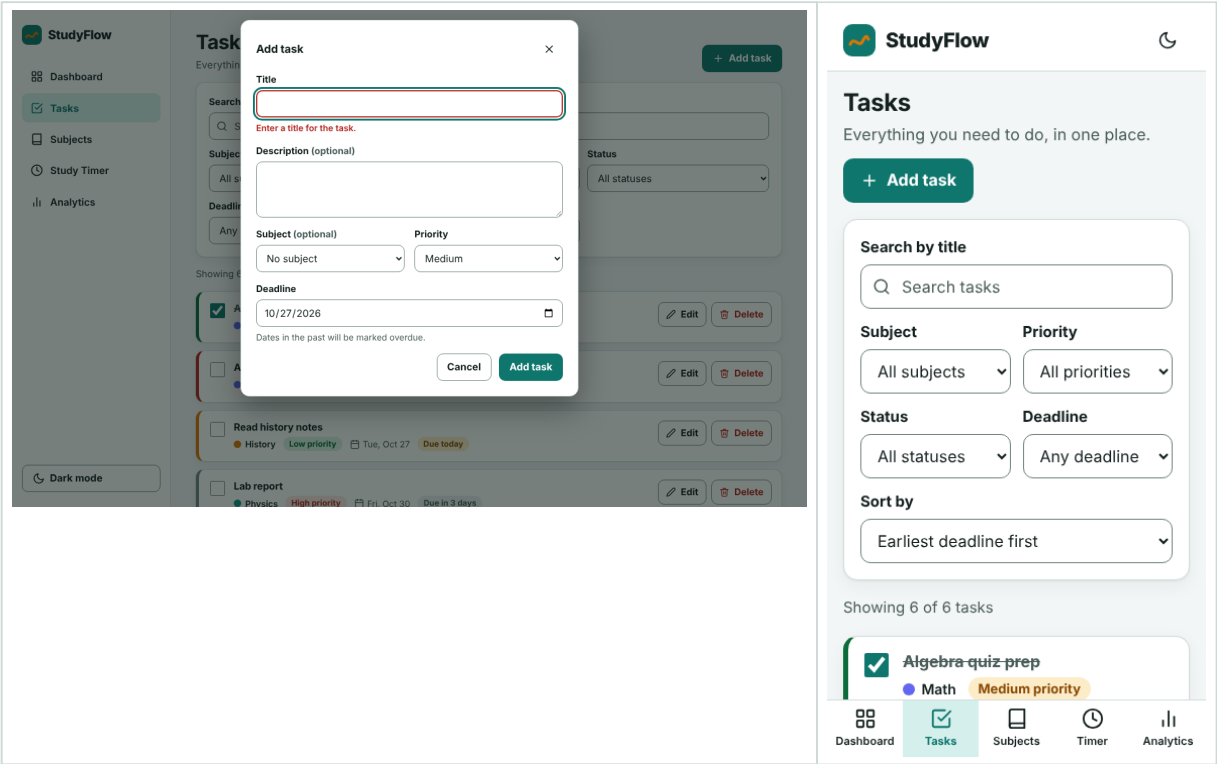
Component	Purpose
`Layout`, `Sidebar`, `MobileHeader`, `BottomNav`, `NavList`, `Brand`	The frame: sidebar on desktop, top bar and bottom bar on small screens, skip link, page title and focus handling.
`ThemeToggle`	Light/dark button.
`PageHeader`, `StatCard`, `EmptyState`, `ProgressBar`, `Badge`	Small building blocks reused across pages.
`TaskForm`, `TaskFilters`, `TaskList`, `TaskItem`	Everything on the Tasks page. `TaskFilters` is a controlled component: it keeps no state and reports changes upward.
`SubjectForm`, `SubjectList`, `SubjectTag`	Subject form with colour picker, the cards, and the coloured label used everywhere.
`DeadlineBadge`, `PriorityBadge`, `DeadlineList`	"Overdue by 2 days", "High priority", and the dashboard list of next deadlines.
`TimerDial`, `DurationForm`, `SessionHistory`	The ring clock, timer length settings, list of finished sessions.
`BarChart`, `SubjectBars`, `TaskSplit`	Charts drawn with HTML/CSS, with text equivalents for screen readers.
`FormField`	Label + input + hint + error, correctly linked for assistive technology.
`Modal`, `ConfirmDialog`	Dialogs on the native `<dialog>` element (focus trapping, Esc to close, dimmed backdrop).
`Toast`, `StorageNotices`, `ErrorBoundary`	Short confirmations; warnings about damaged or unsaveable data; a safety net for crashes.
`Icon`	Inline SVG icons.

7. How each feature works

7.1 Task management

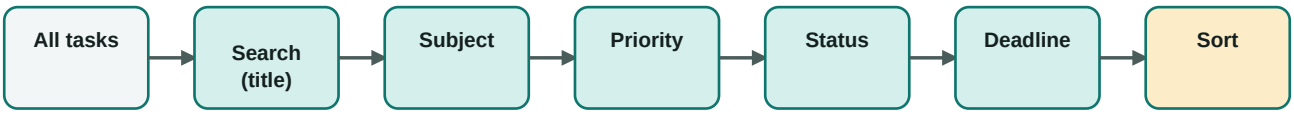
Tasks have a title, optional description, optional subject, priority and deadline. The form validates on Save and tells you exactly what is wrong next to the field:

Field	Rule
Title	Required (spaces only does not count), at most 100 characters.
Description	Optional, at most 500 characters.
Deadline	Required, and must be a real date between 2000 and 2100 (so 30 February is refused).
Priority	Must be High, Medium or Low.
Subject	Optional, but must still exist (protects against a subject deleted in another tab).



The add-task dialog showing a validation error with focus on the invalid field (left), and the Tasks page on a 375 px phone (right).

7.2 Search, filters, sorting



Every step keeps only the tasks that match. A task must pass ALL steps to be shown, then the survivors are sorted.
This is why filters combine: each one narrows the result of the previous one.

All of this happens in one pure function, `filterAndSortTasks`. Filters are: subject (including "No subject"), priority, status, and deadline (overdue, due today, next 7 days, more than 7 days away). Equal deadlines are ordered by priority, then title, so the order never jumps around. When nothing matches, the page says so and

offers **Clear filters**; the button only appears when a filter is actually active.

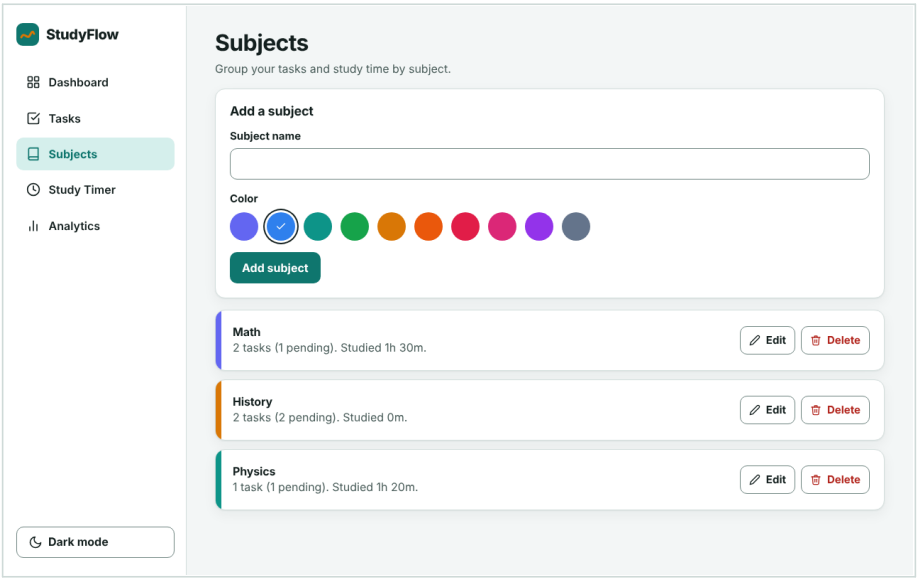
7.3 Deadlines

State	Rule	Shown as
Completed	The task is ticked, even if its date is in the past.	Green "Completed" badge, struck-through title.
Overdue	Not completed and the deadline is before today.	Red "Overdue by N days"; red banner on the Dashboard.
Due today	Not completed and the deadline is today.	Amber highlight and "Due today".
Upcoming	Not completed and the deadline is later.	"Due tomorrow", "Due in N days".

State is never shown with colour alone: each one also has words, so it is clear to people who cannot tell the colours apart. The Dashboard lists the next six pending tasks in date order, overdue ones first.

7.4 Subjects

A subject is a name and a colour. Names are compared ignoring capital letters and extra spaces, so "math", " Math " and "MATH" are all duplicates of "Math". When you edit a subject, it is allowed to keep its own name. Before deleting, a dialog explains how many tasks use it and that they will be kept and shown as "No subject". Study history for a deleted subject also becomes "No subject".



The Subjects page: each card shows its task count and total study time.

7.5 LocalStorage and safe loading

Five keys are used: `studyflow.tasks`, `.subjects`, `.sessions`, `.preferences` and `.timer`. Loading never trusts what it finds:

Situation	What StudyFlow does
Nothing saved yet (first visit)	Starts empty. No warning.
Invalid JSON (for example `{broken`)	Starts empty for that key, shows a notice, and keeps the original text under `studyflow.<key>.backup`.
Valid JSON but the wrong shape	Treated the same as invalid.

Situation	What StudyFlow does
Some items are broken (missing title, impossible date, negative minutes)	Keeps the good items, drops the bad ones, shows a notice, keeps a backup.
An item has a bad priority or colour	Repairs it (medium priority, default colour) instead of dropping it.
Duplicate ids	Keeps the first one.
Browser blocks saving (private mode, storage full)	The app keeps working and shows a red warning that changes will be lost on refresh.
Browser blocks reading	Starts empty, no crash.

7.6 Dashboard statistics

Card	How it is calculated
Total tasks	Number of tasks.
Completed / Pending	Tasks with `completed` true / total minus completed.
Due today	Pending tasks whose deadline equals today's date.
Studied today	Sum of `durationMin` of sessions that finished on today's local date.
Completion	completed / total x 100, rounded. 0% when there are no tasks (no division by zero).
Upcoming deadlines	Pending tasks sorted by date, first 6.

7.7 The Pomodoro timer

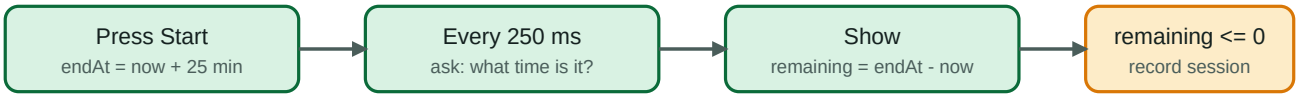
The Pomodoro technique alternates focused work with short breaks. StudyFlow defaults to 25 minutes of focus, 5 of short break, and 15 of long break, which is suggested after every fourth focus session of the day. All three lengths can be changed (focus 1-180, short 1-60, long 1-120 minutes; decimals, blanks and text are refused).

Naive timer (breaks when the tab sleeps)



If the browser pauses the tab for 10 minutes, these ticks never happen and the clock is wrong.

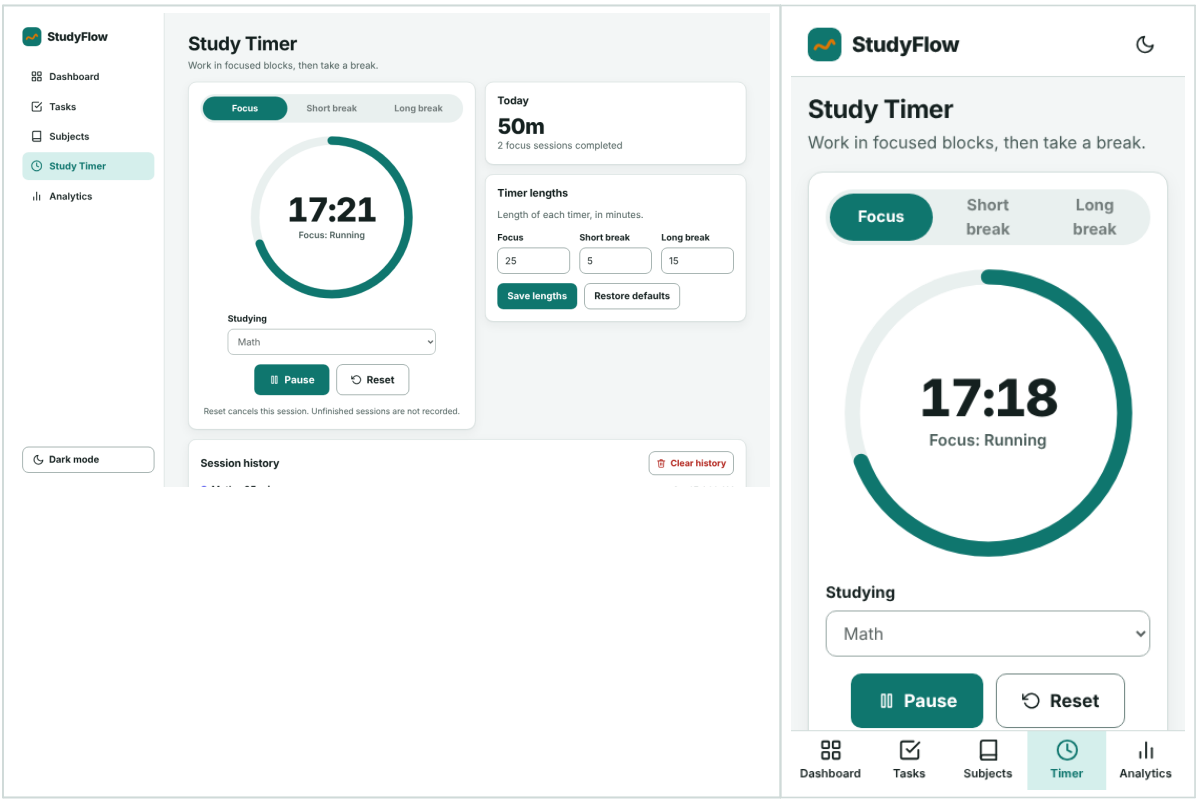
StudyFlow timer (timestamp based)



The interval only refreshes the screen. The truth is always "end time minus real clock", so a sleeping tab, another page, or a reload cannot make the timer drift. The session id is created at Start, so saving the same session twice is ignored (no duplicate records).

- **Start / Pause / Resume / Reset.** Pausing freezes the remaining time; resuming sets a new end time from it. Reset returns to idle and records nothing.
- **While running**, the mode buttons and the subject list are disabled so a session cannot be lost by accident.
- **Subject.** A focus session is attached to the subject chosen when it started.

- **Recording.** When the end time is reached, one session is saved: subject, planned length, and the exact finish time. Breaks are never recorded.
- **No duplicates.** The session id is created at Start. Saving a session whose id already exists does nothing. This also protects against React's development mode, which runs effects twice on purpose.
- **Reload or closed tab.** The running timer is saved. If it finished while the app was closed, it is recorded the next time you open the app, stamped with its real finish time (not "now").



The timer running on desktop (17:21 left) and on a 375 px phone.

7.8 Study tracking

Everything is calculated from the list of recorded sessions: study time today, the last seven days ("weekly", meaning today and the six days before it), time by subject, and the session history (newest first; the first 8 are shown with a "Show all" button; "Clear history" asks for confirmation). Sessions whose subject was deleted are grouped as "No subject".

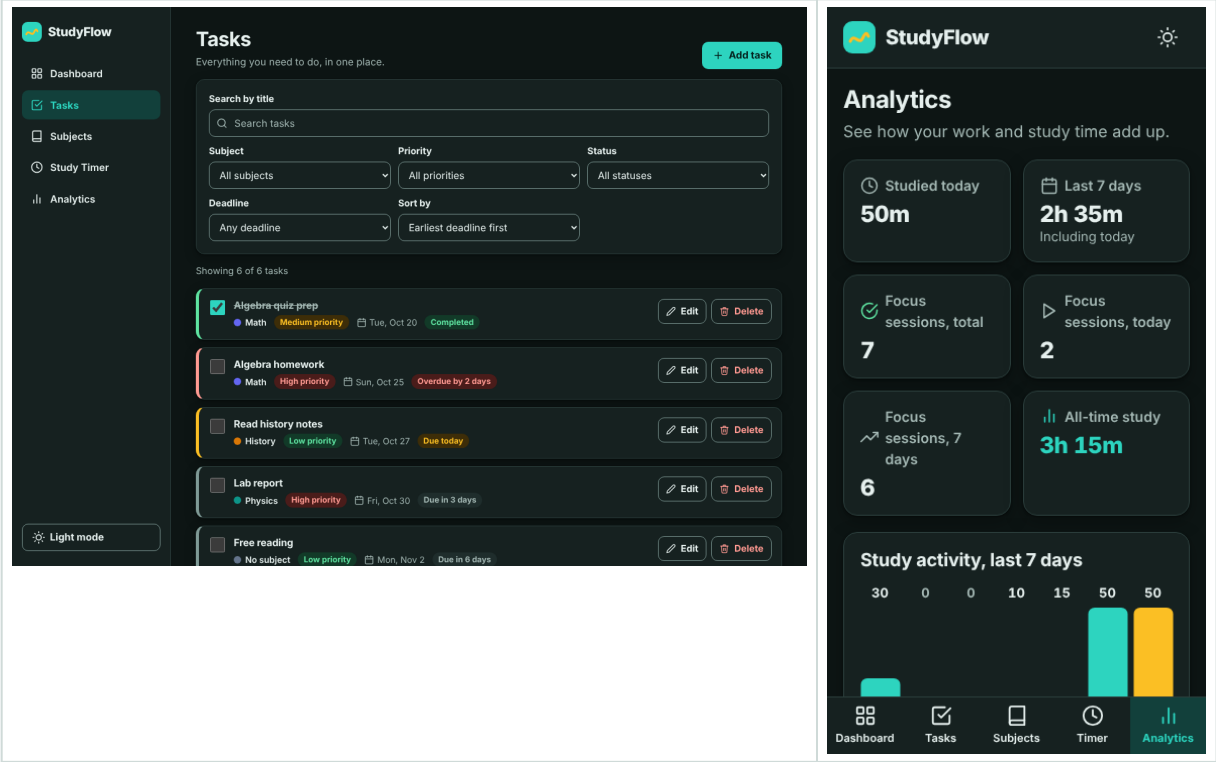
7.9 Analytics

- **Completed vs pending**: one stacked bar (completed, pending, overdue) with a legend showing the exact numbers, plus the completion percentage.
- **Seven-day chart**: one column per day, including days with zero study, oldest first; today is highlighted. A hidden table with the same numbers is provided for screen readers.
- **By subject**: horizontal bars sorted by minutes, in each subject's colour, with session counts.
- **Session counts**: today, last 7 days, and total.
- **Empty data**: each card shows a short explanation and a button to the page where you can create the data. Nothing shows "NaN" or an empty chart frame.

7.10 Light and dark themes

All colours are CSS variables defined in `theme.css`. Dark mode is `data-theme="dark"` on the page, which redefines those variables. The choice is saved in `studyflow.preferences`. If you have never chosen, StudyFlow follows your device setting but does not save that as a choice. A small script in `index.html` applies the theme before the first paint, so there is no flash of the wrong theme.

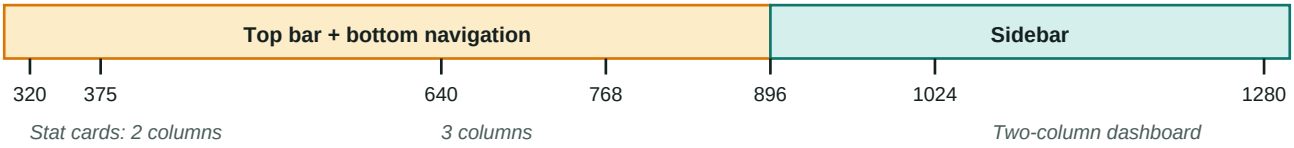
> **Contrast was measured, not guessed**
>
> Every text and background colour pair was checked against the WCAG AA standard (at least 4.5:1 for normal text, 3:1 for borders and graphics) before the stylesheet was written: all 40 pairs passed in both themes. Afterwards, an automated browser test measured the real rendered colour of every visible piece of text on all five pages, in both themes, with a dialog open: none was below the AA minimum (4.5:1, or 3:1 for large text).



Dark theme on desktop (left) and on a phone (right).

7.11 Responsive design

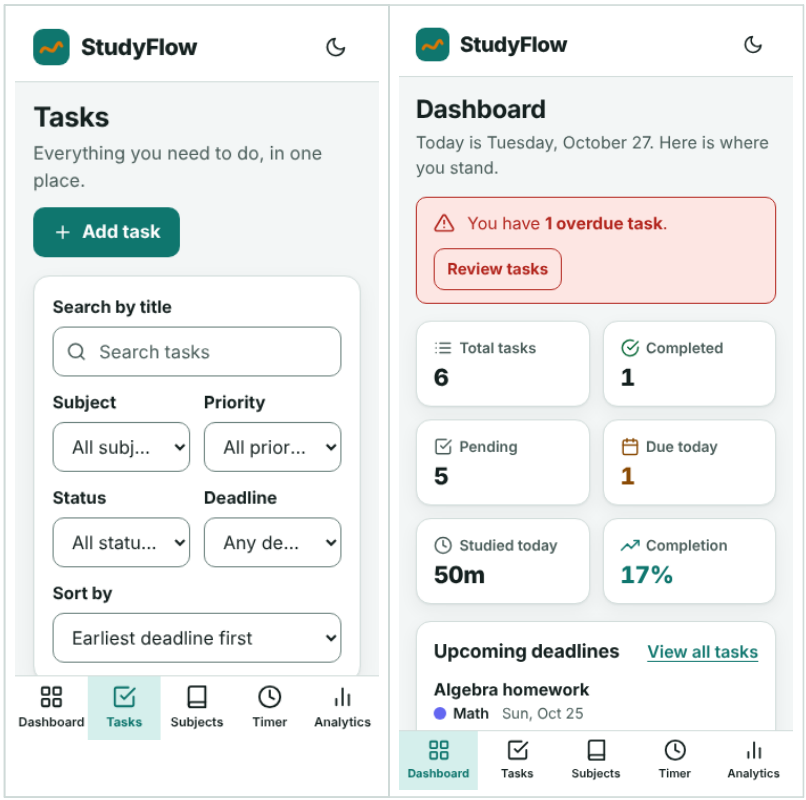
Screen width (px)



Tested at 320, 375, 768, 1024 and 1280. At every width there is no sideways scrolling.

- The app is built **mobile first**: the default CSS is for phones, and media queries add layout for larger screens.
- Phones and tablets get a top bar and a bottom navigation bar, easy to reach with a thumb. From 896 px wide, a sidebar replaces them.
- Grids use `minmax(0, 1fr)` and text wraps with `overflow-wrap`, so long words cannot push the page wider than the screen.
- Inputs use a 16 px font (stops iPhones zooming in), and buttons and fields are at least 40 px tall.

- Dialogs fit the screen and scroll inside themselves on short screens.



The smallest supported width (320 px, left) and 375 px (right). No sideways scrolling.

7.12 Accessibility

Feature	Detail
Semantic HTML	Real buttons, links, lists, headings, `main`, `nav`, `dialog`, `time`, `fieldset`. One `h1` per page and no skipped heading levels.
Forms	Every field has a visible label; errors are linked with `aria-describedby`; invalid fields get `aria-invalid`; focus moves to the first invalid field.
Keyboard	"Skip to main content" is the first Tab stop; dialogs trap focus and close with Esc; focus returns to the button that opened a dialog; visible focus outline everywhere.
Screen readers	Icon-only buttons have text labels; charts have text tables; toasts use `role="status"`; storage failures use `role="alert"`; the page title changes with the page.
Colour	Meaning is never shown by colour alone (overdue says "Overdue by 2 days").
Motion	Animations are turned off for people who ask for reduced motion.

8. Testing and quality

The project rule was: **do not claim a feature is complete until it has been tested.** Below is what was tested, how, and the result. Nothing in this section is an estimate.

Layer	Tool	Result
Unit tests: dates, validation, filtering, statistics, timer, storage	Node built-in test runner (`npm test`)	**74 passed, 0 failed**
Browser tests: whole app, real Chromium	Python + Playwright, with a fake clock for the timer	**28 passed, 0 failed** (production build)
Same browser tests against the dev server	Shows React warnings and StrictMode double-effects	**28 passed, 0 failed**, no console warnings
Stability	Full browser suite run three times in a row	**28/28 each time**
Accessibility audit	axe-core on all 5 pages x 2 themes + dialogs with errors	**0 violations** (after the fix in section 8.2)
Colour contrast	Computed in the browser for every visible text	**0 below the AA minimum** (4.5:1; 3:1 for large text), both themes
Overflow and navigation	320, 375, 768, 1024, 1280 px on all 5 pages	**No horizontal overflow**; correct navigation at each width
Production build	`npm run build`, also from a clean copy with `npm ci`	**Succeeds** (276 KB JS, 18 KB CSS)

8.1 What the browser tests verify

Area	Checks
Task creation	Empty, too-long and missing-deadline errors; focus moves to the invalid field; the error clears while typing; the new task appears with the right data; the default deadline is today.
Editing	Form is pre-filled; saving changes the task; no duplicate is created.
Completion	Ticking and unticking updates the row, the saved data, and `completedAt`.
Deletion	Cancel keeps the task; Delete removes it and the saved data.
Filtering	Every filter alone, several combined, search with odd capitalisation and spaces, nothing matching, Clear filters, both sort orders, and a finished old task is not counted as overdue.
Subjects	Duplicate names (different case and spaces), editing, colours, delete keeps tasks and sessions.
Timer	Start, pause (time frozen for 5 simulated minutes), resume, finish, exactly one record, reset records nothing, breaks record nothing, long break after the 4th session, invalid lengths never saved, countdown survives page changes and reloads, a session that ended while closed is recorded once at its real finish time, same session id is never saved twice.
Study time	Daily, weekly (7-day window excludes day 8), per-subject totals, counts, session order, collapse/expand, clear history.
Persistence	Tasks, subjects, timer lengths and theme survive reload; a second tab sees the data; corrupted, partly broken and blocked storage are survived.
Dates	Midnight rollover turns "due today" into "overdue" without a reload.
Dark mode	Toggle, saved, applied before first paint, explicit light beats a dark device setting, device default not saved as a choice.

8.2 Errors found, and how each was solved

The first full browser run had **4 failures out of 28**. Rather than quietly patching until it went green, each failure was investigated to decide whether the **app** or the **test** was wrong. Several more problems were found afterwards by looking at screenshots and by running against the development server. This table is the honest record.

#	Symptom	Cause	Type	Solution
1	Test error "strict mode violation: resolved to 2 elements"	The test looked for a button called "Clear filters", but the page had two (one in the filter bar, one in the "no results" message).	Test bug	Made the test name which one it means. The app was right.
2	320 px test: subject was not created	The test typed a 37-character name; the limit is 30, so the app correctly refused it.	Test bug	Used a 30-character name. This confirmed the validation works.
3	axe: "Heading levels should only increase by one"	The Tasks page went from an `h1` straight to `h3` task titles.	**Real app bug**	Task titles became `h2`; the visual size is unchanged by CSS. Re-audit: 0 violations.
4	Test said focus "escaped the modal"	With a native `<dialog>`, pressing Tab past the last button hands focus to the browser's own toolbar. The page behind stays locked.	Test too strict	The test now checks that focus never lands on anything behind the dialog (it may be in the dialog or on the browser).

#	Symptom	Cause	Type	Solution
5	Dev server only: first Tab did not land on "Skip to main content"	React's development mode runs effects twice. My "is this the first render?" flag was set on the first run, so the second run moved focus to the page on load.	**Real app bug** (dev only, but fragile)	Replaced the flag with "did the route actually change?", which is correct however many times the effect runs.
6	Test failed right after clicking a nav link	The test read the focused element immediately, before the page change finished.	Test timing	The test now waits for the condition (up to 3 s) instead of reading once.
7	Screenshot: page title hidden under the sticky header on a phone after changing page	Moving focus to the main area also scrolled the page to a position behind the header.	**Real UI bug** (found visually)	Focus without scrolling, then scroll to the top. Verified in a new screenshot.
8	Screenshot: timer length labels wrapped to two lines and misaligned	"Short break (minutes)" was too long for a narrow column.	Real UI polish	Short visible labels, a line of explanation above, and "length in minutes" kept for screen readers.
9	A command that restarted the server ended with exit code 144	My own shell command contained the text it was trying to kill, so it killed itself.	My tooling mistake	Restarted the server in a separate step. No project files were affected.

> **A lesson for beginners**
>
> When a test fails, ask first: "is the app wrong, or is my test wrong?" Items 3, 5, 7 were real bugs that tests or screenshots caught. Items 1, 2, 4, 6 were mistakes in the tests. Changing a test to make it pass is only correct when you can explain why the test was wrong.

8.3 Manual testing checklist

Use this in your own browser. For phone sizes use Chrome DevTools, "Toggle device toolbar". (A fuller version is in the README.)

Tasks

- [ ] Add a task with every field filled. It appears.
- [ ] Try an empty title, a 101-character title, an empty deadline: each shows an error beside its field.
- [ ] Edit it; no duplicate is created.
- [ ] Tick it done and back; the Dashboard counts change.
- [ ] Delete it: Cancel keeps it, Delete removes it.

Filters

- [ ] Create 4+ tasks with different subjects, priorities, dates (one past, one today).
- [ ] Search part of a title in any capitalisation.
- [ ] Use each filter alone, then several together.
- [ ] Pick filters that match nothing: message and working Clear filters button.

Subjects

- [ ] Adding "math" when "Math" exists is refused.
- [ ] Edit name and colour; tasks show the change.
- [ ] Delete a subject with tasks: tasks stay, shown as "No subject".

Deadlines / Dashboard

- [ ] Due today is highlighted and counted.
- [ ] A past unfinished task says "Overdue by N days" and shows the red banner.
- [ ] Upcoming deadlines are in date order.

Timer (set Focus to 1 minute)

- [ ] Start, Pause, Resume, Reset work.
- [ ] Change page and return: time is still right. Reload: countdown continues.
- [ ] A finished session adds exactly one history entry.
- [ ] Reset mid-session records nothing.
- [ ] Lengths 0, 2.5, empty, 999 are refused.

Analytics

- [ ] Totals equal the sum of the history.
- [ ] Seven-day chart ends with today.
- [ ] After `localStorage.clear()` and reload, every card shows a helpful empty message.

Persistence / theme

- [ ] Reload keeps everything including theme.
- [ ] Run `localStorage.setItem("studyflow.tasks","{broken")` and reload: notice shown, app works.

Responsive / keyboard

- [ ] At 320 and 375 px: no sideways scroll, forms and dialogs fit.
- [ ] Sidebar on desktop, bottom bar on phones.
- [ ] Keyboard only: first Tab is "Skip to main content"; Esc closes dialogs.

9. Run, publish and deploy

9.1 Installation and running

You need **Node.js 18 or newer**. Check with `node -v`; download it from nodejs.org if it is missing.

```
cd studyflow
npm install        # download dependencies (once)
npm run dev        # development server -> open http://localhost:5173
npm test           # run the 74 unit tests
npm run build      # create the production site in dist/
npm run preview    # open the production build locally
```

Common errors

`npm: command not found`: Node.js is not installed (or the terminal was opened before installing). Install it and open a new terminal.

`Port 5173 is already in use`: another dev server is running. Close it, or Vite will offer the next free port.

White page after deploying to a sub-folder: make sure `vite.config.js` still contains `base: './'`.

Data "disappeared": LocalStorage belongs to one browser and one address (`localhost:5173` and `localhost:4173` are different). Open the same address in the same browser.

9.2 Optional: browser tests

```
npm run build && npm run preview                  # terminal 1
pip install playwright && playwright install chromium
python e2e/studyflow_e2e.py                       # terminal 2
```

9.3 Git and GitHub

Git records the history of your project; GitHub stores a copy online (and is what recruiters will look at).

```
git config --global user.name  "Your Name"        # once per computer
git config --global user.email "you@example.com"

cd studyflow
git init
git add .
git commit -m "Initial commit: StudyFlow"
git branch -M main
# create an EMPTY repository on github.com, then:
git remote add origin https://github.com/YOUR-USERNAME/studyflow.git
git push -u origin main
```

Everyday routine: `git status`, then `git add .`, then `git commit -m "what you changed"`, then `git push`. The included `.gitignore` already keeps `node_modules` and `dist` out of the repository. Make small commits with clear messages; the history is part of your portfolio.

9.4 Deployment

`npm run build` produces a `dist/` folder of plain files. Any static host can serve it, and no server settings are needed.

Host	Steps
Netlify (easiest)	Run `npm run build`, go to app.netlify.com/drop, and drag the `dist` folder onto the page. Or connect the GitHub repo: build command `npm run build`, publish directory `dist`.
Vercel	Import the GitHub repository. Vite is detected automatically (output folder `dist`).
GitHub Pages	Settings -> Pages -> Source: "GitHub Actions". Add the workflow file shown in the README (`.github/workflows/deploy.yml`), commit and push. Your site will be at `https://YOUR-USERNAME.github.io/studyflow/`. **Note:** this workflow follows GitHub's standard pattern but was not run while building this project; check the Actions tab after your first push.

Before every deployment: `npm test`, `npm run build`, `npm run preview`, and click through the preview.

10. Known limitations

Being clear about what the project does not do is part of professional work. These are real limits of the current version:

Limitation	Detail
Data stays in one browser	No accounts and no sync. A different browser or device, or clearing site data, starts empty. There is no export/import yet.
Two tabs at once	Both write to the same LocalStorage keys; the last to save wins. Use one tab at a time.
No alert when the timer ends	No sound or browser notification. A session that ends while the app is closed is recorded correctly on the next visit, but you are not told when it ended. Keep the tab open.
Reset discards the session	Only completed focus sessions count; partial ones are not recorded.
"Weekly" is a rolling 7 days	Today plus the previous six days, not Monday to Sunday.
Deleting a subject loses its study attribution	Its sessions become "No subject". The minutes still count in the totals.
Device clock	Dates use the computer's date and time zone. Changing the system date changes "today" and "overdue".
No undo	Deleting a task or clearing history asks for confirmation but cannot be undone afterwards.
Browser coverage	**Tested only in Chromium** (desktop browser, including emulated 320 and 375 px phones). Not tried on Safari, Firefox or physical phones. Not tested with a real screen reader; the automated accessibility audit passes, but that is not a substitute.
Storage size	LocalStorage holds about 5 MB per site: thousands of tasks, not unlimited. The app warns if saving fails.
GitHub Pages workflow	Provided but not executed in the build environment.

Ideas for next steps

- Export and import data as a JSON file (also a backup).
- A sound or notification when the timer ends.
- Recurring tasks and a calendar view.
- A weekly study goal with a progress ring.
- Run the tests automatically in GitHub Actions on every push.
- Learn a router library (React Router) and a charting library, and compare them with the hand-made versions.

11. Presenting it, and a glossary

How to talk about this project

- **The problem:** students lose track of deadlines and cannot see how they spend study time.
- **The technical highlights:** a timestamp-based timer that stays accurate in background tabs; safe loading of untrusted saved data; every statistic calculated from real data; accessibility measured with tools rather than assumed; 74 unit tests and 28 browser tests.
- **A trade-off you can explain:** no router, state or chart libraries, in order to learn the fundamentals; a team project might choose libraries instead.
- **What you would do next:** export/import, notifications, and a backend with accounts for syncing between devices.

Glossary

Term	Meaning
Component	A function that returns a piece of the screen.
Props	The inputs a component receives from its parent.
State	Data a component remembers between redraws; changing it redraws the screen.
Hook	A function starting with `use` that lets components use React features (`useState`, `useEffect`...).
Context	A way to share data with many components without passing props through every level.
Pure function	A function whose output depends only on its input and which changes nothing outside itself.
Immutable update	Changing data by making a new copy instead of editing the original.
Controlled input	A form field whose value is owned by React state.
LocalStorage	A small store of text key/value pairs kept in the browser.
Sanitise	Check and clean data that came from outside before trusting it.
Timestamp	A moment in time as a number (milliseconds since 1 January 1970).
StrictMode	A React development helper that deliberately runs some code twice to expose bugs.
Semantic HTML	Using the HTML element that matches the meaning of the content (`nav`, `button`, `main`).
WCAG AA	A widely used accessibility standard; includes minimum colour contrast.
Breakpoint	The screen width at which the layout changes.
Build	Turning your source code into small optimised files ready to host.
Repository (repo)	A project folder tracked by Git.
Commit	A saved snapshot of your project in Git.